

程序的执行

计算机所有功能都通过执行程序完成，程序由指令序列构成。现代计算机最突出的特点之一是采用“存储程序”的工作方式，因此，程序被启动后，计算机能自动逐条取出程序中的指令并执行。数据通路在控制器产生的控制信号控制下完成特定的指令功能。数据通路和控制器是中央处理器（Central Processing Unit，简称 CPU）中最基本的部件。本书中有时将中央处理器简称为处理器。

一个处理器能够处理的指令集合及其涉及的指令格式、寻址方式、操作数的定义、操作数所存放的寄存器以及存储器的相关规定就是处理器的指令集体系结构（ISA）。不同的处理器系列具有不同的指令集体系结构。例如，Intel 的 x86 系列处理器具有相同或相互兼容的 ISA，称之为 IA-32，ARM 系列处理器具有相同或相互兼容的 ISA。不同厂商只要按照相同的 ISA 来设计制造处理器芯片，那么这些处理器芯片就是相互兼容的，可以在机器中互换。

按照 ISA 的复杂程度来分，有复杂指令集计算机（Complex Instruction Set Computer，简称 CISC）和精简指令集计算机（Reduced Instruction Set Computer，简称 RISC）两种类型。IA-32 属于前者，其相应的数据通路比较复杂，为简化机器指令执行原理的讲解，本章采用 RISC 风格的 MIPS 处理器作为模型机。

本章主要介绍指令的执行过程、CPU 的基本功能和基本组成、数据通路的工作原理和设计方法以及流水线方式下指令的执行过程。

5.1 程序执行概述

5.1.1 程序及指令的执行过程

从第 3 章和第 4 章介绍的有关机器级代码的生成与表示可以看出，指令按顺序存放在存储空间连续单元中，正常情况下，指令按其存放顺序执行，遇到需要改变程序执行流程的情况，用相应的转移类指令（包括无条件转移指令、条件转移指令、调用指令和返回指令等）来改变程序执行流程。即将执行的指令所在存储单元的地址由程序计数器给出。CPU 取出并执行一条指令的时间称为指令周期，不同指令的指令周期可能不同。

例如，对于第 4 章 4.4.2 节中的例子，其链接生成的可执行目标文件的 .text 节中的 main 函数包含的指令序列如下。

```

1 08048380 <main>:
2 8048380:      55                push    %ebp
3 8048381:      89 e5            mov     %esp,%ebp
4 8048383:      83 e4 f0         and     $0xfffffffff0,%esp
5 8048386:      e8 09 00 00 00   call    8048394 <swap>
6 804838b:      b8 00 00 00 00   mov     $0x0,%eax
7 8048390:      c9                leave   %eax
8 8048391:      c3                ret

```

可以看出,指令按顺序存放在地址 0x08048380 开始的存储空间中,每条指令的长度可能不同,如 push、leave 和 ret 指令各占一个字节,第 3 行的 mov 指令占两个字节,第 4 行 and 指令占三个字节,第 5 行和第 6 行指令都占 5 个字节。每条指令对应的 0/1 序列的含义有不同的规定,如“push %ebp”指令为 55H=01010101B,其中高 5 位 01010 为 push 指令操作码,后三位 101 为 EBP 的编号,“leave”指令为 C9H=11001001B,没有显式操作数,8 位都是指令操作码。指令执行的顺序是:第 2~5 行指令按顺序执行,第 5 行指令执行后跳转到 swap 函数执行,执行完 swap 函数后回到第 6 行指令执行,然后顺序执行到第 8 行指令,执行完第 8 行指令后,再转到另一处开始执行。

CPU 为了能完成指令序列的执行,它必须解决以下一系列问题:如何判定每条指令有多长?如何判定指令操作类型、寄存器编号、立即数等?如何区分第 3 行和第 6 行 mov 指令的不同?如何确定操作数是在寄存器中还是在存储器中?一条指令执行结束后如何正确地读取到下一条指令?

通常,CPU 执行一条指令的大致过程如图 5.1 所示,分成取指令、指令译码、计算源操作数地址并取操作数、执行数据操作、计算目的操作数地址并存结果、计算下一条指令地址这几个步骤。

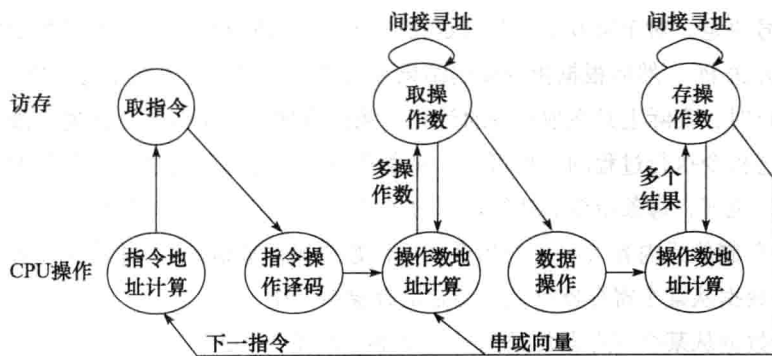


图 5.1 指令执行过程

① 取指令。即将执行的指令的地址总是在程序计数器（PC）中,因此,取指令的操作就是从 PC 所指出的存储单元中取出指令送到指令寄存器（IR）。例如,对于上述 main 函数的执行,刚开始时,PC（即 IA-32 中的 EIP）中存放的是首地址 0x08048380,因此,CPU 根据 PC 的值取到一串 0/1 序列送 IR,可以每次总是取最长指令字节数,假定最长指令是 4 个字节,即 IR 为 32 位,此时,从 0x08048380 开始取 4 个字节到 IR 中,也即,将 55H、89H、E5H 和 83H 送到 IR 中。

② 对 IR 中的指令操作码进行译码。不同指令其功能不同，即指令涉及的操作过程不同，因而需要不同的操作控制信号。例如，上述第 6 行“mov \$0x0,%eax”指令要求将立即数 0x0 送寄存器 EAX 中；而上述第 3 行“mov %esp,%ebp”指令则要求从寄存器 ESP 中取数，然后送寄存器 EBP 中。因而，CPU 应该根据不同的指令操作码译出不同的控制信号。例如，对取到 IR 中的 5589E583H 进行译码时，可根据对最高 5 位 (01010) 的译码结果得到 push 指令的控制信号。

③ 源操作数地址计算并取操作数。根据寻址方式确定源操作数地址计算方式，若是存储器数据，则需要一次或多次访存，例如，当指令为间接寻址或两个操作数都在存储器的双目运算时，就需要多次访存；若是寄存器数据，则直接从寄存器取数后转到下一步进行数据操作。

④ 执行数据操作。在 ALU 或加法器等运算部件中对取出的操作数进行运算。

⑤ 目的操作数地址计算并存结果。根据寻址方式确定目的操作数地址计算方式，若是存储器数据，则需要一次或多次访存（间接寻址时）；若是寄存器数据，则在进行数据操作时直接存结果到寄存器。

如果是串操作或向量运算指令，则可能会并行执行或循环执行第③~⑤步多次。

⑥ 指令地址计算并将其送 PC。顺序执行时，下条指令地址的计算比较简单，只要将 PC 加上当前指令长度即可，例如，当对 IR 中的 5589E583H 进行操作码译码时，得知是 push 指令，指令长度为一个字节，因此，指令译码生成的控制信号会控制使 PC 加 1 (即 $0x08048380 + 1$)，得到即将执行的下条指令的地址为 0x08048381。如果译码的是转移类指令时，则需要根据条件标志、操作码和寻址方式等确定下条指令地址。

对于上述过程的第①步和第②步，所有指令的操作都一样；而对于第③~⑤步，不同指令的操作可能不同，它们完全由第②步译码得到的控制信号控制。也即指令的功能由第②步译码得到的控制信号决定。对于第⑥步，若是定长指令字，处理器会在第①步取指令的同时计算出下条指令地址并送 PC，然后根据指令译码结果和条件标志决定是否在第⑥步修改 PC 的值，因此，在顺序执行时，实际上是在取指令时计算下条指令地址，第⑥步什么也不做。

根据对上述指令执行过程的分析可知，每条指令的功能总是通过对以下四种基本操作进行组合来实现的，也即，每条指令的执行可以分解成若干个以下的基本操作。

- ① 读取某存储单元内容（可能是指令或操作数或操作数地址），并将其装入某个寄存器。
- ② 把一个数据从某个寄存器存储到给定的存储单元中。
- ③ 把一个数据从某个寄存器传送到另一个寄存器或者 ALU。
- ④ 在 ALU 中进行某种算术运算或逻辑运算，并将结果传送到某个寄存器。

5.1.2 CPU 的基本功能和组成

CPU 的基本职能是周而复始地执行指令，上一节介绍的机器指令执行过程中的全部操作都是由 CPU 中的控制器控制执行的。随着超大规模集成电路技术的发展，更多的功能逻辑被集成到 CPU 芯片中，包括 cache、MMU、浮点运算逻辑、异常和中断处理逻辑等，因而 CPU 的内部组成越来越复杂，甚至可以在一个 CPU 芯片中集成了多个处理器核。不管 CPU 多复杂，它最基本的部件是数据通路和控制部件。控制部件根据每条指令功能的不同生成对数据通路的控

制信号，并正确控制指令的执行流程。

为了在教学上遵循由简到难的原则，我们首先从 CPU 最基本的组成开始了解。CPU 的基本功能决定了 CPU 的基本组成，图 5.2 所示是 CPU 的基本组成原理图。

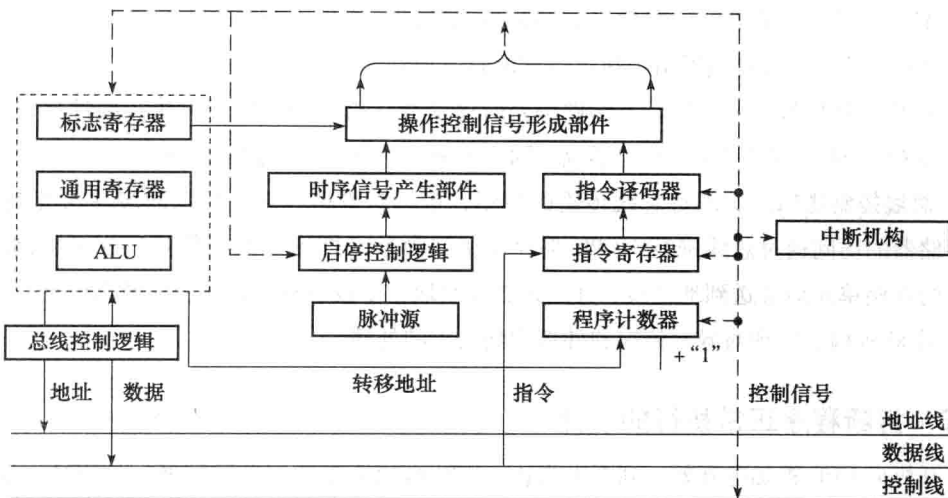


图 5.2 CPU 基本组成原理图

图 5.2 中的地址线、数据线和控制线并不属于 CPU，构成系统总线的这三组线主要用来使 CPU 与 CPU 外部的部件（主要是主存储器）交换信息，交换的信息包括地址、数据和控制三类，分别通过地址线、数据线和控制线进行传送，这里，数据信息包含指令，也即数据和指令都看成是数据，因为对总线和存储器来说，指令和数据在形式上没有区别，而且数据和指令的访存过程也完全一样。除了地址和数据（包括指令）以外的所有信息都属于控制信息。地址线是单向的，由 CPU 送出 CPU 之外的地址，用于指定需要访问的指令或数据所在的存储单元地址。

图 5.2 所示的数据通路非常简单，只包括最基本的执行部件，如 ALU、通用寄存器和状态寄存器等，其余都是控制逻辑或与其密切相关的逻辑，主要包括以下几个部分。

① 程序计数器 (PC)。PC 又称指令计数器或指令指针寄存器 (IP)，用来存放即将执行指令的地址。正常情况下，指令地址的形成有两种方式：

- 顺序执行时， $PC + "1"$ 形成下条指令地址（这里的“1”是指一条指令的字节数）。在有的机器中，PC 本身具有“+1”计数功能，也有的机器借用运算部件完成 $PC + "1"$ 。
- 需要改变程序执行顺序时，通常会根据转移类指令提供的信息生成转移目标指令的地址，并将其作为下条指令地址送 PC。每个程序开始执行之前，总是把程序中第一条指令的地址送到 PC 中。

② 指令寄存器 (IR)。IR 用以存放现行指令。上文提到，每条指令总是先从存储器取出后才能在 CPU 中执行，指令取出后存放在指令寄存器中，以便送指令译码器进行译码。

③ 指令译码器 (ID)。ID 对指令寄存器中的操作码部分进行分析解释，产生相应的译码信号提供给操作控制信号形成部件，以产生控制信号。

④ 脉冲源及启停控制逻辑。脉冲源产生一定频率的脉冲信号作为整个机器的时钟脉冲，它是 CPU 时序的基准信号。启停控制逻辑在需要时能保证可靠地开放或封锁时钟脉冲，控制时序信号的发生与停止，并实现对机器的启动与停机。

⑤ 时序信号产生部件。该部件以时钟脉冲为基础，产生不同指令对应的周期、节拍、工作脉冲等时序信号，实现机器指令执行过程的时序控制。

⑥ 操作控制信号形成部件。该部件综合时序信号、指令译码信号和执行部件反馈的条件标志（如 CF、SF、ZF 和 OF）等，形成不同指令的操作所需要的控制信号。

⑦ 总线控制逻辑。实现对总线传输的控制，包括对数据和地址信息的缓冲与控制。CPU 对于存储器的访问通过总线进行，CPU 将存储访问命令（即读写控制信号）送到控制线，将要访问的存储单元地址送到地址线，并通过数据线取指令或者与存储器交换数据信息。

⑧ 中断机构。实现对异常情况和外部中断请求的处理。

5.1.3 打断程序正常执行的事件

从开机后 CPU 被加电开始，到断电为止，CPU 自始至终就一直重复做一件事情：读出 PC 所指存储单元的指令并执行它。每条指令的执行都会改变 PC 中的值，因而 CPU 能够不断地执行新的指令。

正常情况下，CPU 按部就班地按照程序规定的顺序一条指令接着一指令执行，或者按顺序执行，或者跳转到转移类指令设定的转移目标指令执行，这两种情况都属于正常执行顺序。

当然，程序并不总是能按正常顺序执行，有时 CPU 会遇到一些特殊情况而无法继续执行当前程序。例如，以下事件可能会打断程序的正常执行。

- 对指令操作码进行译码时，发现是不存在的“非法操作码”，因此，CPU 不知道如何实现当前指令而无法继续执行。
- 在访问指令或数据时，发现“段错误（Segmentation fault）”或“缺页（Page fault）”，因此，CPU 没有获得正确的指令和数据而无法继续执行当前指令。
- 在 ALU 中运算的结果发生溢出，或者整数除法指令的除数为 0，因此，CPU 发现运算结果不正确而无法继续执行程序。
- 在执行指令过程中，CPU 外部发生了采样计时时间到、网络数据包到达网络适配器、磁盘完成数据读写等外部事件，要求 CPU 中止当前程序的执行，转去执行专门的外部事件处理程序。

因此，CPU 除了能够正常地不断执行指令以外，还必须具有程序正常执行被打断时的处理机制，这种机制被称为异常控制，也被称为中断机制，CPU 中相应的异常和中断处理逻辑称为中断机构，如图 5.2 中所示。

计算机中很多事件的发生都会中断当前程序的正常执行，使 CPU 转到操作系统中预先设定的与所发生事件相关的处理程序执行。有些事件处理完后可回到被中断的程序继续执行，此时相当于执行了一次过程调用；有些事件处理完后则不能回到原被中断的程序继续执行。所有这些打断程序正常执行的事件被分成两大类：内部异常和外部中断。

1. 内部异常

内部异常 (exception) 是指由 CPU 内部的异常引起的意外事件。根据其发生的原因又分为硬故障中断和程序性异常。硬故障中断是由于硬连线路出现异常而引起的,如电源掉电、校验线路错等;程序性异常是由 CPU 执行某个指令而引起的发生在 CPU 内部的异常事件,如除数为 0、溢出、断点、单步跟踪、寻址错、访问超时、非法操作码、堆栈溢出、缺页、地址越界(段错误)等。

2. 外部中断

程序执行过程中,若外设完成任务或发生某些特殊事件,例如,打印机缺纸、定时采样计数时间到、键盘缓冲区已满、从网络中接收到一个信息包、从磁盘读入了一块数据等,设备控制器会向 CPU 发中断请求,要求 CPU 对这些情况进行处理。通常,每条指令执行完后,CPU 都会主动去查询有没有中断请求,有的话,则将下条指令地址作为断点保存,然后转到用来处理相应中断事件的“中断服务程序”执行,结束后回到断点继续执行。这类事件与执行的指令无关,由 CPU 外部的 I/O 子系统发出,所以称为I/O 中断或外部中断 (interrupt),需要通过外部中断请求线向 CPU 请求。

有关内部异常和外部中断更详细的内容参见第 7 章。显然,内部异常和外部中断都会破坏流水线的执行,引起指令流水线的阻塞。指令流水线中的异常和中断处理是非常具有挑战性的工作,已超出本书讨论的范围,请参考其他相关资料。

5.2 数据通路基本结构和工作原理

机器指令的执行是在数据通路中完成的,通常将指令执行过程中数据所经过的路径(包括路径上的部件)称为数据通路。ALU、通用寄存器、状态寄存器、cache、MMU、浮点运算逻辑、异常和中断处理逻辑等都是指令执行过程中数据流经的部件,都属于数据通路的一部分。通常把数据通路中专门进行数据运算的部件称为执行部件 (execution unit) 或功能部件 (function unit)。数据通路由控制部件进行控制。

5.2.1 数据通路基本结构

指令执行所用到的元件有两类:组合逻辑元件 (也称操作元件) 和时序逻辑元件 (也称状态元件或存储元件)。连接这些元件的方式有两种:总线方式和分散连接方式。数据通路就是由操作元件和存储元件通过总线或分散方式连接而成的进行数据存储、处理和传送的路径。

1. 操作元件

组合逻辑元件的输出只取决于当前的输入。若输入一样,其输出也一样。组合电路的定时不受时钟信号的控制,所有输入信号到达后,经过一定的逻辑门延迟,输出端的值被改变,并一直保持其值不变,直到输入信号改变。数据通路中常用的组合逻辑元件有多路选择器 (MUX)、加法器 (Adder)、算术逻辑部件 (ALU)、译码器 (Decoder) 等,其符号表示如图 5.3 所示。有关这些组合逻辑元件的功能和结构请参看附录。

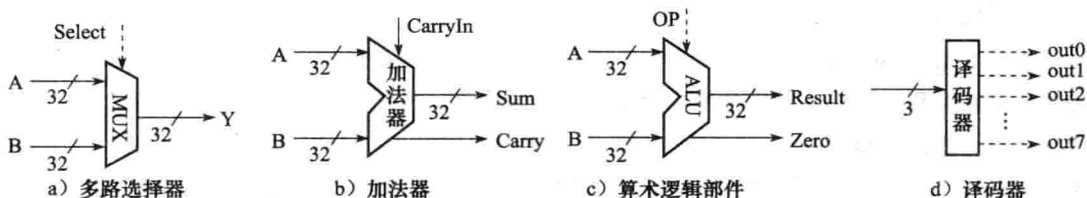


图 5.3 数据通路中的常用组合逻辑元件

图 5.3 中虚线表示控制信号，多路选择器需要控制信号 Select 来确定选择哪个输入被输出；加法器不需要控制信号进行控制，因为它的操作是确定的；算术逻辑部件需要有操作控制信号 OP，由它确定 ALU 进行哪种操作；译码器通过对指令操作码进行译码，输出译码后得到的控制信号 out0、out1、…、out7，译码器无需控制信号进行控制。

2. 状态元件

状态元件具有存储功能，输入状态在时钟控制下被写到电路中，并保持电路的输出值不变，直到下一个时钟到达。输入端状态由时钟决定何时被写入，输出端状态随时可以读出。最简单的状态单元是 D 触发器，有时钟输入 Clk、状态输入端 D 和状态输出端 Q。图 5.4 是 D 触发器的定时示意图。

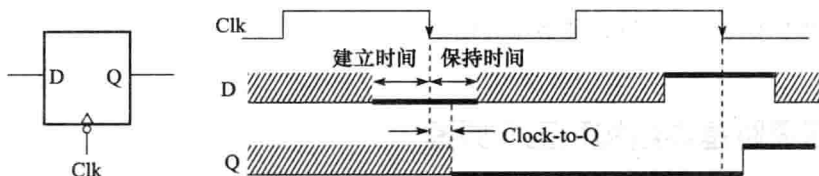


图 5.4 D 触发器定时示意图

图 5.4 中的 D 触发器采用时钟下降沿（负跳变）触发，要使输出状态能正确地随着输入状态改变，必须满足以下时间约束：① 在时钟下降沿到来前的一定时间内，输入端 D 的状态必须稳定有效，这段时间被称为 建立时间 (Setup Time)；② 在时钟下降沿到来后的一定时间内，输入端 D 的状态必须继续保持稳定不变，这段时间被称为 保持时间 (Hold Time)。在上述两个约束条件满足的情况下，经过时钟下降沿到来后的一段延迟时间，输出端 Q 的状态改变为输入端 D 的状态，并一直保持不变，直到下个时钟到来。通常，将触发边沿开始到输出端 Q 的状态改变为止的时间称为 Clock-to-Q 时间 (“Clock-to-Q”)，也称为触发器的 锁存延迟 (Latch Prop)，这段时间远小于保持时间。

数据通路中的寄存器是一种典型的状态存储元件，由 n 个 D 触发器可构成一个 n 位寄存器。根据功能和实现方式的不同，有各种不同类型的寄存器。

5.2.2 数据通路的时序控制

指令执行过程中的每个操作步骤都有先后顺序，为了使计算机能正确执行指令，CPU 必须按正确的时序产生操作控制信号。由于不同指令对应的操作序列长短不一，序列中各操作的执行时间也不相同，因此，需要考虑用怎样的时序方式来控制。

早期计算机通常采用机器周期、节拍和脉冲三级时序对数据通路操作进行定时控制。一个指令周期可分为取指令、读操作数、执行并写结果等多个基本工作周期，称为机器周期。有取指令、存储器读、存储器写、中断响应等不同类型的机器周期。每个机器周期的实际时间长短可能不同，例如，存储器读或存储器写周期比 CPU 中的操作时间长得多。所以，机器周期的宽度通常以主存工作周期为基础来确定。一个机器周期内要进行若干步动作。例如，存储器读周期有送地址、发读命令、检测数据有无准备好、取数据等。因此，有必要将一个机器周期再划分成若干节拍，每个动作在一个节拍内完成。为了产生操作控制信号并使某些操作能在一拍时间内配合工作，常在一个节拍内再设置一个或多个工作脉冲。

现代计算机中，已不再采用三级时序系统，机器周期的概念已逐渐消失。整个数据通路中的定时信号就是时钟信号，一个时钟周期就是一个节拍。

如图 5.5 所示，数据通路可看成由组合逻辑元件（即操作元件）和状态元件（即存储元件）交替组合而成，即数据通路的基本结构为“……状态元件—操作元件（组合逻辑）—状态元件……”。

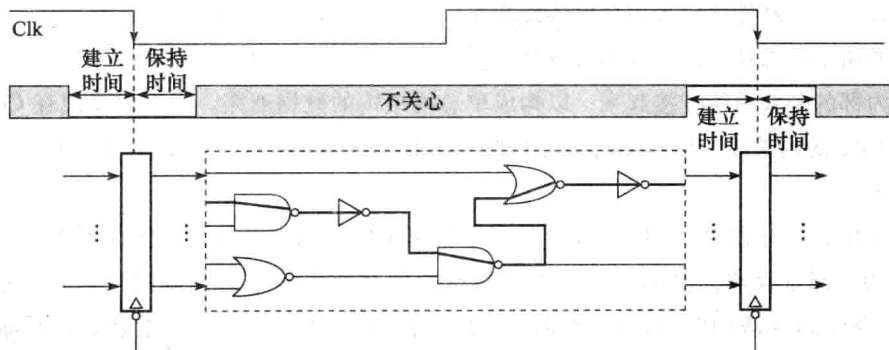


图 5.5 数据通路和时钟周期

只有状态元件能存储信息，所有操作元件都须从状态元件接收输入，并将输出写入状态元件中。所有状态元件在同一个时钟信号控制下写入信息。假定采用下降沿触发方式，则所有状态元件在时钟下降沿到来时开始写入信息，经过触发器的锁存延迟（即 Clk-to-Q 时间）后输出开始有效。假定每个时钟的下降沿是一个时钟周期的开始时刻，则一个时钟周期内整个处理过程如下：经过 Clk-to-Q 时间，前一个时钟周期内生成的信号被写入状态元件，并输出到随后的操作元件进行处理，经过若干级门延迟，得到的处理结果被送到下一级状态元件的输入端，然后再稳定一段时间（建立时间）才能开始下个时钟周期，并在时钟信号到达后还要保持一段时间（保持时间）。

假定所有各级操作元件中最长操作延迟时间为 Longest Delay，考虑时钟偏移（clock skew）^①，根据上述分析可知，数据通路的时钟周期为：

$$\text{Cycle Time} = \text{Clk-to-Q 时间} + \text{Longest Delay} + \text{建立时间} + \text{时钟偏移}$$

① 时钟偏移：由于器件工艺和走线延迟等原因造成的同步系统中时钟信号的偏差，这种时间偏差使得时钟信号不能同时到达不同的状态元件而导致同步定时错误，所以需要在时钟周期中增加时钟偏移时间来避免这种错误。也有人把 clock skew 翻译为时钟扭斜或时钟歪斜。

5.2.3 数据通路基本工作原理

从1946年冯·诺依曼及其同事在普林斯顿高级研究院开始设计存储程序计算机（被称为IAS计算机，它是后来通用计算机的原型）以来，数据通路的结构发生了较大变化，从IAS计算机中CPU内部的分散连接结构，到基于单总线、双总线或三总线的总线式CPU结构，再到基于简单流水线和超标量/动态调度流水线CPU结构，直至近几年又出现了多核CPU结构。特别是近几年，执行程序的底层硬件还出现了CPU+GPU或CPU+MIC协处理器等新的执行架构。不管执行机器指令的数据通路有多么复杂，其基本工作原理都是相通的。为简化问题，本节采用简单的总线式数据通路和基本流水线数据通路来讲解数据通路的基本工作原理，有关现代计算机数据通路设计的一些复杂问题，请参看其他相关资料。

1. 单总线数据通路

通用计算机的原型是IAS计算机，它所使用的数据通路中，除了主存M外，主要有累加器AC、乘商寄存器MQ、指令寄存器IR、程序计数器PC等。它采用了累加器型指令系统，因而数据通路较简单，部件之间采用分散连接方式，即部件之间通过专用的连接线互连。

数据通路中的部件之间也可通过总线方式连接。如图5.6所示，可将ALU及所有寄存器通过一条内部的公共总线连接起来，以构成单总线结构的数据通路。因为此总线在CPU内部，所以称为CPU的内总线，请不要把它与CPU外部的用于连接CPU、存储器和I/O模块的系统总线相混淆。

图5.6中，寄存器 $R_0 \sim R_{(n-1)}$ 是程序员可见的通用寄存器，而寄存器Y和Z是对程序员透明的内部寄存器，仅用作某些指令执行期间存放中间结果的临时寄存器。指令寄存器IR和指令译码器是CPU内部控制器的主要部件。系统总线经由存储器数据寄存器（Memory Data Register，简称MDR）和存储器地址寄存器（Memory Address Register，简称MAR）连到CPU。

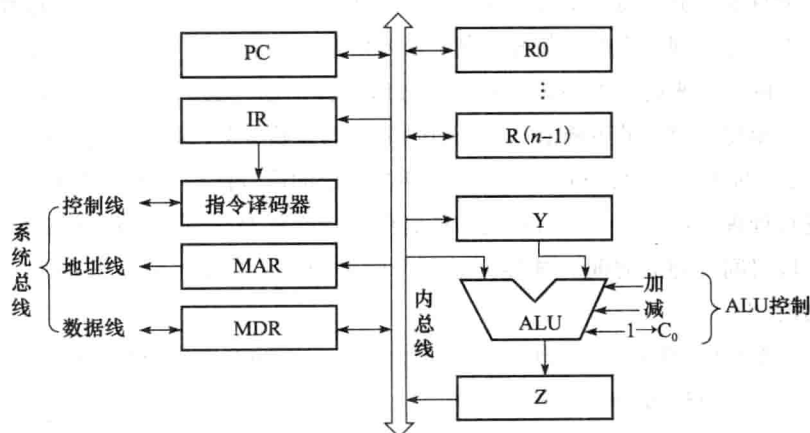


图 5.6 单总线数据通路

在图5.6所示的单总线数据通路结构中，组成所有指令功能的四种基本操作过程说明如下。

(1) 在通用寄存器之间传送数据

总线是一组共享的传输信号线，它不能存储信息，某一时刻也只能有一个部件能把信息送到总线上。在图 5.6 所示的单总线数据通路中，连到 CPU 内总线上的各个部件之间通过内总线传送数据。源寄存器将信息送到内总线上，经过内总线上一定的时间延迟，被传送到目的寄存器并被存储在目的寄存器中。通常在寄存器和内总线之间有两个控制信号： R_{in} 和 R_{out} 。当 $R_{in} = 1$ 时，控制将内总线上的信息存到寄存器 R 中；当 $R_{out} = 1$ 时，控制寄存器 R 将信息送到内总线上。图 5.7 给出了寄存器中的一位触发器和内总线相连时的控制电路和控制信号，对于一个由 n 个触发器构成的 n 位寄存器，其原理是一样的。

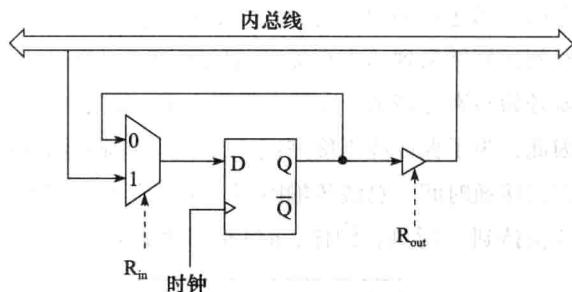


图 5.7 一位寄存器的输入/输出控制

在图 5.7 中，D 触发器的数据输入端连到一个二路选择器的输出，当控制信号 $R_{in} = 1$ 时，选择内总线上的信息输出到 D 触发器的输入端，当时钟信号的触发边沿到达时，被存入触发器；当 $R_{in} = 0$ 时，触发器的值保持不变。触发器的输出端通过一个三态门与内总线相连，当控制信号 $R_{out} = 1$ 时，三态门被打开，触发器的输出被送到总线上；当 $R_{out} = 0$ 时，三态门的输出端呈高阻态，触发器与内总线断开。例如，在图 5.6 所示的数据通路中，要将寄存器 R0 的内容传送到寄存器 Y，则对应的控制信号取值为： $R0_{out} = 1$ ， $Y_{in} = 1$ ，其余寄存器的 R_{in} 和 R_{out} 信号都为 0，通常称取值为 1 的 R_{in} 和 R_{out} 信号为有效控制信号，在描述控制信号取值时，只写出有效控制信号，例如，上述情况下写为“ $R0_{out}$ ， Y_{in} ”。

(2) 完成算术或逻辑运算

ALU 是一个没有记忆功能的组合逻辑电路，若要进行正确的运算，必须将两个操作数都送到 ALU 的输入端，并在正确的 ALU 控制信号的控制下进行。ALU 控制信号可以是“加 (add)”、“减 (sub)”、“与 (and)”、“或 (or)”、“传送 (mov)”、“取反 (not)”、“取负 (neg)”等。在图 5.6 所示的数据通路中，Y 寄存器存放其中一个操作数，另一个操作数被置于内总线上。运算结果被临时存放在寄存器 Z 中。因此，要实现操作：“ $R[R3] \leftarrow R[R1] + R[R2]$ ”，即寄存器 R1 的内容与 R2 的内容相加，结果送寄存器 R3，则有效控制信号如下：

① $R1_{out}$ ， Y_{in} ② $R2_{out}$ ，add，(Z_{in})③ Z_{out} ， $R3_{in}$

以上三步不能同时执行，因为任何时刻只能将一个寄存器的内容送到内总线，任何一步结束时结果要送到某个寄存器的输入端（即保存到寄存器）。因此，该操作需要三个时钟周期（节拍）。

在上述第②步加法操作中，控制信号 $R2_{out}$ 被置 1 后，R2 的输出三态门被接通，数据沿内总线传送到 ALU 的输入端，并在 add 信号的控制下，在 ALU 中与已经在另一个输入端的数据（即 Y 中的内容）做加法运算，最后将结果写入 Z（实际上，在图 5.6 所示的数据通路中，控制信号 Z_{in} 不是必需的，因为 ALU 的输出只有一个出口，即寄存器 Z。通常情况下，无需控制

信号就可直接将 ALU 的结果送 Z 寄存器)。为了能正确实现这一步操作, $R2_{out}$ 、 add 和 Z_{in} 信号必须在一定的时间内保持有效。

如图 5.8 所示, 由于指令译码线路等控制逻辑的延迟, 控制信号总是在时钟信号开始一段时间之后才有效, 我们把这段时间记为 $Clk\text{-}to\text{-}Signal$, 它的时延一定大于锁存延迟 $Clk\text{-}to\text{-}Q$, 因此, 在控制信号开始有效的 t_0 时刻, 寄存器 R2 的输出已经有效, 此时, $R2_{out}$ 开始作用于三态门, 到达 t_1 时刻三态门打开, R2 的内容被送到内总线上, 经过总线传输和 ALU 延迟, 在 t_3 时刻运算结果到达寄存器 Z 的输入端, 再经过一段建立时间, 在 t_4 时刻稳定, 下个时钟到来后就开始写寄存器 Z。为了正确写入 Z, 运算结果还必须在 Z 的输入端继续稳定一段保持时间。因此, 为了保证能正确完成 ALU 运算并将运算结果正确写入 Z 寄存器, $R2_{out}$ 必须至少保持三态门接通时间、总线传输时间、ALU 延迟、寄存器 Z 的建立和保持时间之和, 即 $R2_{out}$ 必须持续保持到 t_5 时刻, 同样, add 信号也必须持续保持到 t_5 时刻。

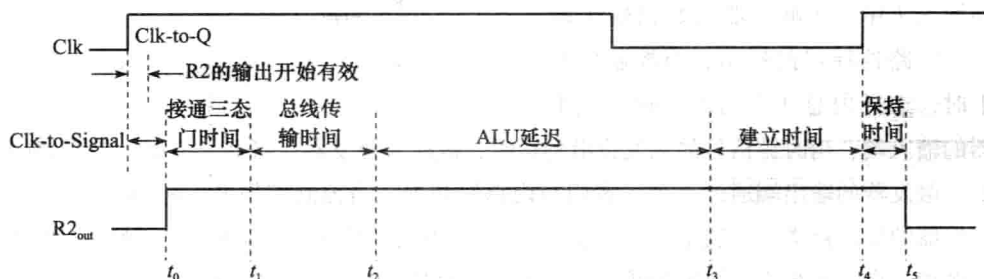


图 5.8 ALU 操作控制信号的定时

(3) 从内存读取一个字 (指令或数据或数据的地址)

早期的计算机, 其 CPU 和主存之间采用“异步”方式进行通信。对于读操作, 首先, CPU 送出一个读信号 (read) 来启动一次主存读操作, 然后等待主存完成读操作。主存一旦完成读操作 (即把数据放到 MDR 中), 就向 CPU 发回一个存储功能完成 (Memory Function Completed, 简称 MFC) 信号, 使 CPU 结束等待状态, 并继续执行指令。CPU 在等待期间, 每个时钟到来时都会采样 MFC 信号, 若检测到 MFC 信号有效, 则表明存储器已将数据读出。为了使 CPU 能从执行指令状态转入等待状态, 需要有一个控制信号, 这里用 WMFC (Wait MFC) 表示该控制信号。

假定需要在某条指令中实现操作: $R[R2] \leftarrow M[R[R1]]$, 该操作的含义是, 将寄存器 R1 所指的主存单元内容装入寄存器 R2, 则在图 5.6 所示的数据通路中完成该操作的有效控制信号序列如下:

- ① $R1_{out}$, MAR_{in}
- ② read, WMFC
- ③ MDR_{out} , $R2_{in}$

第①步通过内总线将 R1 的内容送到 MAR 的输入端, 操作在一个时钟周期内完成。在时钟边沿到来后经过一个锁存延迟 $Clk\text{-}to\text{-}Q$, MAR 输入端的地址被送到系统总线上。第②步在 CPU 发出 read 命令的同时, 通过 WMFC 控制信号使 CPU 转入等待状态, 处于等待状态的时间取决

于所用存储器的速度。通常,从主存读取一个字的时间比在 CPU 内部进行一个运算所用时间要长得多,需要多个时钟周期。因此,这一步不能在一个时钟周期内完成。第③步当 CPU 采样到 MFC 信号有效后,就直接将 MDR 中的内容通过内总线送到 R2,这一步操作在一个时钟周期内完成。

目前,由于主存基本上采用 SDRAM 芯片(同步的动态存储器芯片),所以主存与 CPU 之间大多采用“同步”方式进行通信。同步方式下,存储器总是在读信号 read 发出后的固定几个时钟周期内准备好数据,因而 CPU 不必等待主存发回 MFC 信号,也即,同步方式下第②步不需要 WMFC 信号。

(4) 把一个字(数据)写入主存

操作 $M[R[R2]] \leftarrow R[R1]$ 的含义是,将寄存器 R1 的内容写入寄存器 R2 所指的主存单元中。在图 5.6 所示的数据通路中完成该操作的有效控制信号序列如下:

① $R1_{out}, MDR_{in}$

② $R2_{out}, MAR_{in}$

③ write, WMFC

第③步和上述读内存操作的第②步一样,通常要有多个时钟周期。此外,若主存采用像 SDRAM 芯片这样的同步 DRAM 芯片,则不需要 WMFC 信号。

如果第①步和第②步两个传送不使用相同的物理通道,例如,不送到同一组总线上,则可同时执行。显然,在图 5.6 所示的单总线结构中,这是不允许的。

例 5.1 某计算机字长 16 位,采用 16 位定长指令字结构,部分数据通路结构如图 5.9 所示。假设 MAR 的输出一直处于使能状态。加法指令“ADD (R1), R0”的功能为 $M[R[R1]] \leftarrow M[R[R1]] + R[R0]$ 。

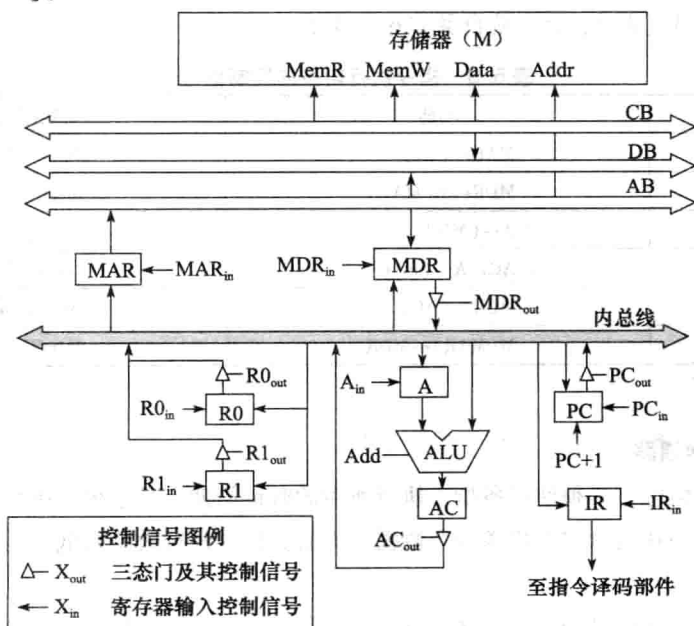


图 5.9 例 5.1 中的数据通路

表 5.1 给出了上述指令取指和译码阶段每个节拍（时钟周期）的功能和有效控制信号，请按表中描述方式列出指令执行阶段每个节拍的功能和有效控制信号，并说明需要多少个节拍。

表 5.1 取指令阶段的控制信号

时钟	功能	有效控制信号
C1	$MAR \leftarrow (PC)$	PC_{out}, MAR_{in}
C2	$MDR \leftarrow M(MAR)$	MemR
	$PC \leftarrow (PC) + 1$	$PC + 1$
C3	$IR \leftarrow (MDR)$	MDR_{out}, IR_{in}
C4	指令译码	无

解 题目给出的取指令阶段的控制信号中，存储器读控制信号 MemR 有效时，并没有伴随出现 WMFC 控制信号，所以，应该将 CPU 和存储器之间看成是“同步”通信方式。加法指令“ADD(R1), R0”的执行阶段每个节拍的功能和有效控制信号如表 5.2 所示。

表 5.2 指令执行阶段的控制信号

时钟	功能	有效控制信号
C5	$MAR \leftarrow (R1)$	$R1_{out}, MAR_{in}$
C6	$MDR \leftarrow M(MAR)$	MemR
	$A \leftarrow (R0)$	$R0_{out}, A_{in}$
C7	$AC \leftarrow A + (MDR)$	MDR_{out}, Add
C8	$MDR \leftarrow (AC)$	AC_{out}, MDR_{in}
C9	$M(MAR) \leftarrow MDR$	MemW

从表 5.2 可看出，在 C6 节拍中同时进行了存储器读和寄存器间传送操作，这样，该指令的执行阶段共有 5 个节拍。当然，也可将存储器读和寄存器间传送操作安排在不同的节拍内进行，这样得到表 5.3。此时，执行阶段共有 6 个节拍。

表 5.3 指令执行阶段的控制信号

时钟	功能	有效控制信号
C5	$MAR \leftarrow (R1)$	$R1_{out}, MAR_{in}$
C6	$MDR \leftarrow M(MAR)$	MemR
C7	$A \leftarrow (MDR)$	MDR_{out}, A_{in}
C8	$AC \leftarrow A + (R0)$	$R0_{out}, Add$
C9	$MDR \leftarrow (AC)$	AC_{out}, MDR_{in}
C10	$M(MAR) \leftarrow MDR$	MemW

2. 三总线数据通路

为提高计算机性能，必须使每条指令执行所用的时钟周期数尽量少。单总线数据通路中一个时钟周期内只允许在内总线上传送一个数据，因而其指令执行效率很低。为此，可采用多总线数据通路结构。

图 5.10 给出了三总线数据通路结构示意图。所有通用寄存器在一个双口寄存器堆（dual-port register file）中。允许两个寄存器的内容同时输出到 A 总线和 B 总线。

和单总线结构相比,多总线结构在执行指令时所需要的步骤大为减少。例如,假定三操作数指令“op R1, R2, R3”的功能为 $R[R3] \leftarrow R[R1] \text{ op } R[R2]$, 则可用内总线 A 和 B 传送源操作数, 内总线 C 传送目的操作数。源总线 A 和 B 与目的总线 C 之间的连接通过 ALU 实现。假定所需操作通过 ALU 一次就可完成, 那么, 该三操作数指令的执行可在一个时钟周期内完成, 所需的有效控制信号为: “ $R1_{\text{outA}}, R2_{\text{outB}}, \text{op}, R3_{\text{inC}}$ ”, 其含义是, 将 R1 内容送内总线 A 的同时, 将 R2 内容送内总线 B, 同时控制在 ALU 中进行相应的 op 操作, 并将 ALU 的输出送到内总线 C 上。

该数据通路中, 若将一个寄存器内容传送到另一个寄存器, 则需要通过 ALU 来完成, 只要控制 ALU 进行“直送 (mov)”操作即可。像图 5.6 所示单总线结构中的 Y 和 Z 这样的临时寄存器, 在多总线结构中可以不要, 这是因为 ALU 的输入通路分别是内总线 A 和 B, 输出通路为内总线 C, 三者无冲突, 而在

图 5.6 所示的单总线数据通路中, 如果缺少了 Y 或 Z, 则 ALU 的输入操作数和输出结果中必定有两个数据同时被送到同一个内总线上, 因而会发生总线数据冲突。

目前, 几乎所有 CPU 都采用流水线方式执行指令, 而采用上述单总线或三总线方式连接的数据通路很难实现指令的流水执行。为了更好地理解 CPU 设计技术, 下面简单介绍一下在流水线数据通路中指令执行的基本工作原理。

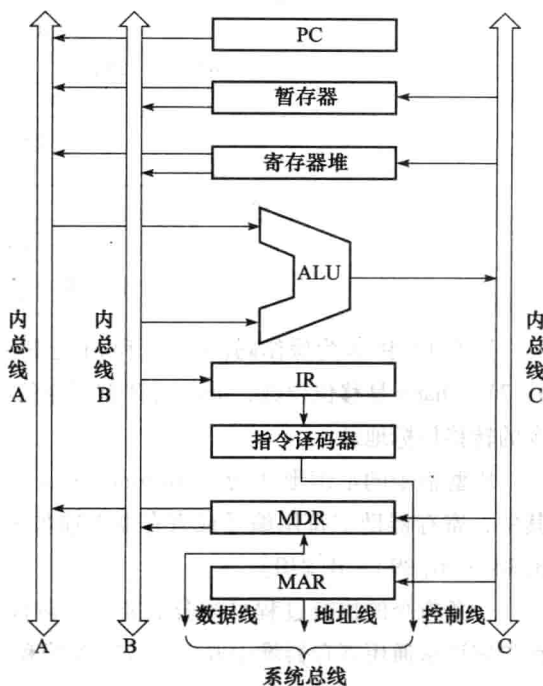


图 5.10 三总线数据通路

5.3 流水线方式下指令的执行

上一节介绍了总线方式数据通路的基本工作原理, 显然, 在总线方式数据通路中, 一条指令的执行需要多个时钟周期才能完成, 而且指令的执行都是串行的。串行方式下, CPU 总是在执行完一条指令后才取出下条指令执行。这种串行方式没有充分利用执行部件的并行性, 因而指令执行效率低。与现实生活中的许多情况一样, 指令的执行也可以采用流水线方式, 即将多条指令的执行相互重叠起来, 以提高 CPU 执行指令的效率。

5.3.1 指令流水线的基本原理

为叙述方便起见, 我们以一个简单的指令系统 MIPS 为例来对指令流水线工作原理进行介绍。MIPS 指令字为 32 位固定长度, 指令格式仅三种, 分别是 R 型、I 型和 J 型, 每种类型的格式如图 5.11 所示。

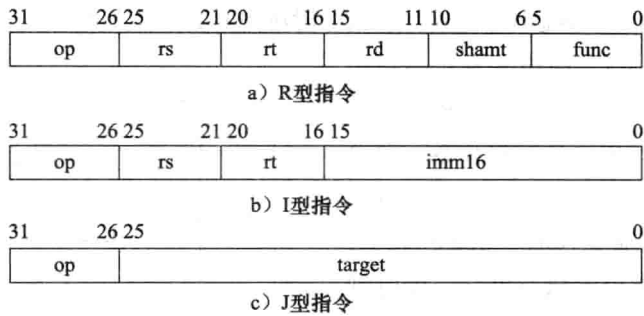


图 5.11 MIPS 指令格式

图 5.11 中 op 为操作码，rs、rt 和 rd 是寄存器编号，MIPS 中有 32 个 32 位寄存器，编号为 0~31，shamt 是移位位数，func 是 R 型指令的功能码，imm16 是 16 位立即数，target 是跳转指令的转移目标地址。

R 型指令的汇编形式为“op(func) rd, rs, rt”，功能为 $R[rd] \leftarrow R[rs] \text{ op(func) } R[rt]$ ，其中，寄存器助记符用编号或寄存器名前加 \$ 的形式表示，例如，“sub \$8, \$9, \$10”表示 $R[8] \leftarrow R[9] - R[10]$ 。

一条指令的执行过程可被分成若干个阶段。例如，对于 MIPS 指令“sub \$8, \$9, \$10”，首先应该从通用寄存器堆中取出 9 号寄存器和 10 号寄存器的内容；然后将取出的这两个数分别送到 ALU 的两个输入端，并控制 ALU 做减法；最后，将 ALU 的输出送到 8 号寄存器的输入端并保持一段时间，这个过程称为将结果写入寄存器。由此看出，一条指令的执行过程中，每个阶段都在相应的功能部件中完成。如果将各阶段看成相应的流水段，则指令的执行过程就构成了一条指令流水线。例如，假定一条指令流水线由如下 5 个流水段组成。

取指令（IF）：根据 PC 的值从存储器取出指令。

指令译码（ID）：产生指令执行所需的控制信号。

取操作数（OF）：读取存储器操作数或寄存器操作数。

执行（EX）：对操作数完成指定操作。

写回（WB）：将操作结果写入存储器或寄存器。

进入流水线的指令流，由于后一条指令的第 i 步与前一条指令的第 $i+1$ 步同时进行，从而使一串指令总的完成时间大为缩短。如图 5.12 所示，在理想状态下，完成 4 条指令的执行只用了 8 个时钟周期，若是非流水线方式的串行执行处理，则需要 20 个时钟周期。

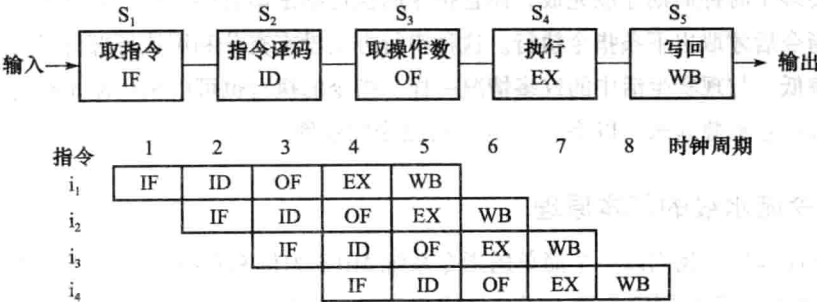


图 5.12 一个 5 段指令流水线

从图 5.12 可看出,理想情况下,每个时钟周期都有一条指令进入流水线;从第 6 时钟周期开始,每个时钟周期都有一条指令完成。由此可以看出,理想情况下每条指令的时钟周期数(即 CPI)都为 1。为了更加清楚地了解流水线的执行效率,下面用一个例子来比较流水线数据通路和单周期数据通路的指令执行情况。

假定指令系统中最复杂的指令需要用以下 5 个阶段完成操作。① 取指: 200ps; ② 译码和读操作数: 50ps; ③ ALU 操作: 100ps; ④ 读存储器: 200ps; ⑤ 结果写寄存器: 50ps。不考虑控制单元、PC 访问、信号传递等延迟,那么,这条指令的总执行时间为 $200 + 50 + 100 + 200 + 50 = 600\text{ps}$ 。

在单周期数据通路中,所有指令都在一个时钟周期内完成,即 $\text{CPI} = 1$,因此这种处理器的时钟周期就是最复杂指令的指令周期。在单周期数据通路中,所有指令的执行都是先在一个组合逻辑组成的操作元件中完成规定的所有操作,然后将得到的结果送到由触发器构成的状态元件(即寄存器)中,因此单周期数据通路由组合逻辑和寄存器组成。图 5.13 给出了单周期数据通路结构以及相应的指令执行过程。

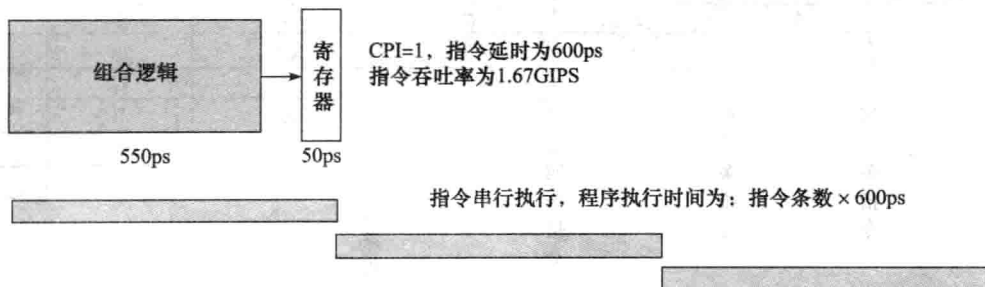


图 5.13 单周期数据通路结构示意图及相应指令执行过程

如图 5.13 所示,在单周期数据通路中,对于上述给定的指令系统,最复杂指令执行时,前面四个阶段包括取指令、译码和读操作数、ALU 操作以及读存储器,总共是 550ps,这些操作都可以在组合逻辑电路中完成,最后一步将结果写入寄存器。实际上是将前面四步得到的结果送到寄存器的输入端并稳定 50ps。按照最复杂指令所用延时来设计单周期数据通路,能够保证每条指令的执行都能在一个时钟周期内正确完成。每条指令的执行都是分两步:首先,在组合逻辑中进行处理,需要最多 550ps 的时间;然后,将组合逻辑处理结果送到寄存器输入端并稳定 50ps。时钟周期就是最复杂指令的指令周期,在此,就是 $550 + 50 = 600\text{ps}$ 。指令吞吐率为 $1/(600 \times 10^{-12}) = 1.67\text{GIPS}$ 。

流水线数据通路的设计原则是:指令流水段个数以最复杂指令所用的功能段个数为准;流水段的长度以最复杂功能段的操作所用时间为准。考虑实现上述指令系统的流水线数据通路,最复杂指令有 5 个功能段,最复杂功能段的时间为 200ps。按照指令流水线设计原则,该流水线应有 5 个流水段,每个流水段由一个组合逻辑和一个寄存器组成,寄存器用来保存对应组合逻辑处理的结果。因此,组合逻辑延时为 200ps,寄存器延时为 50ps,如图 5.14 所示。

流水线数据通路按照最复杂指令所用时间来划分流水段。例如,在图 5.14 中,虽然组合

逻辑 B 的执行时间只需 50ps，组合逻辑 C 只需 100ps，但是，为了使得指令流水线能够顺畅流动，必须按照最长流水段进行划分，也即流水线处理器的时钟周期按照最长流水段时间来确定。这样，组合逻辑 B 和 C 中得到的结果会在不到 200ps 的时间内就先被送到寄存器的输入端，一直到当前时钟周期结束、下个时钟周期到来，再开始进入下一级流水段继续执行。这样，各段组合逻辑可以完全并行工作，在最长 200ps 的延时后，各组合逻辑的处理结果被送到各自对应的寄存器输入端并稳定 50ps，因此，在不考虑控制单元、PC 访问、信号传递等延迟的情况下，时钟周期为 $200 + 50 = 250\text{ps}$ 。

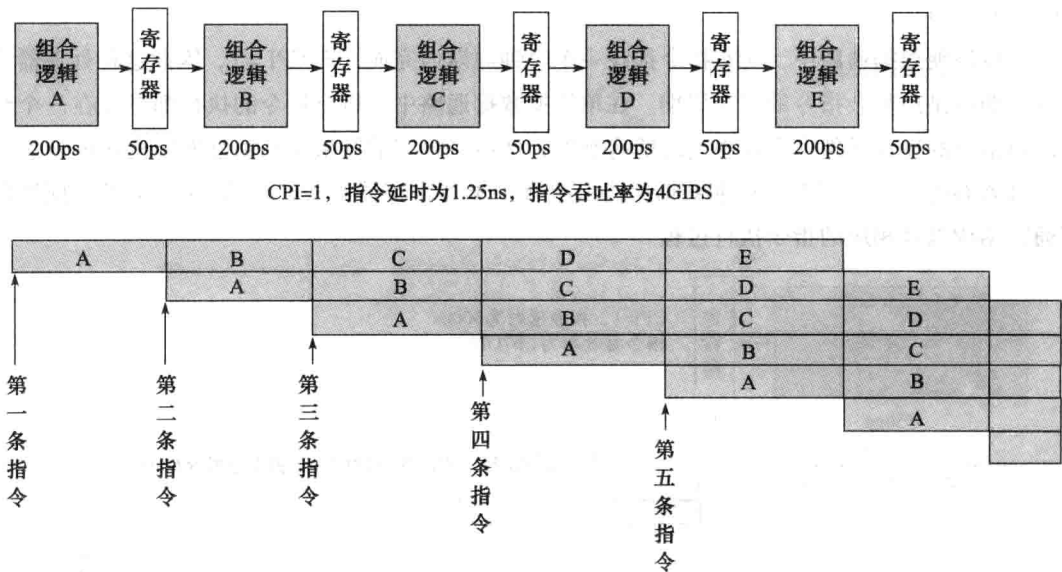


图 5.14 流水线数据通路结构示意图及相应指令执行过程

比较图 5.13 和图 5.14 可以发现，图 5.13 中的组合逻辑应该对应图 5.14 中的组合逻辑 A、B、C 和 D 的总合，也即图 5.14 中流水线数据通路多了一个组合逻辑 E，这是因为有些指令在组合逻辑 D 的执行阶段读出存储单元的值后，可能还要进行像多路选择甚至 ALU 运算等操作。为了使流水线数据通路的时钟周期尽量短，需要再增加一个流水段。不然的话，需要把多路选择器或 ALU 等电路加到组合逻辑 D 中，使得组合逻辑 D 无法在 200ps 内完成相应的功能，这样时钟周期就会大于 250ps，从而影响指令吞吐率。因此，每条指令都被划分为 5 个流水段，每条指令的延时为 $250\text{ps} \times 5 = 1.25\text{ns}$ ，比单周期数据通路串行执行方式增加了 650ps，这其中一部分时间是由于每个流水段后增加了一个写寄存器操作造成的，这是额外开销，流水线划分得越细这个额外开销越大。此外，由于流水段并不是均匀划分，有些流水段中的组合逻辑（如组合逻辑 B、C 和 E）本来能够更早完成功能，但为了使流水线流畅，必须让其等待，因而浪费了一些时间。可以看出，流水线方式并不能缩短一条指令的执行时间，流水线也不是划分得越细越好，而是划分得越均匀越好。在划分比较均匀的情况下，可以大大提高整个程序执行的指令吞吐率。对于图 5.14 中的指令流水线，在理想情况下，因为每个时钟周期有一条指令执行完，所以指令吞吐率为 $1/(250 \times 10^{-12}) = 4\text{GIPS}$ ，吞吐率大约是单周期数据通路的 $4/1.67 = 2.4$ 倍。

5.3.2 适合流水线的指令集特征

按流水线方式执行指令的关键是, 将指令集中所有指令都分成相同数目的功能段, 并让每个功能段的执行时间都相同, 因而, 指令流水段个数以最复杂指令所用的功能段个数为准, 流水段的长度以最复杂功能段的操作所花时间为准。具有什么特征的指令集有利于实现指令流水线呢?

首先, 指令长度应尽量一致。这样, 有利于简化取指令和指令译码操作, 例如, MIPS 架构的指令都是 32 位, 每条指令占 4 个存储单元, 因此, 每次取指令都是读取 4 个单元, 且下址计算也方便, 只要 $PC + 4$ 即可; 而 IA-32 指令从 1 字节到 15 字节不等, 使取指令部件极其复杂, 取指令所用时间也长短不一, 而且也不利于指令译码。

其次, 指令格式应尽量规整, 尽量保证源寄存器的位置相同。这样, 有利于在指令未译码时就可取寄存器操作数。例如, MIPS 指令格式中, 源操作数寄存器 *rs* 和 *rt* 的位置总是分别固定在指令的第 25 ~ 21 位和第 20 ~ 16 位, 因此, 在指令译码的同时就可读取寄存器 *rs* 和 *rt* 的内容。若像 IA-32 那样源操作数寄存器的位置随指令不同而不同, 那么必须先译码后才能确定指令中各寄存器编号的位置, 这样, 从寄存器取数的工作就不能提前到和译码操作同时进行。

第三, 采用 load/store 型指令风格。所谓 load/store 型指令风格, 是指指令集中只有 load 指令和 store 指令能访问存储器, 其他指令一律不能访问, load 指令用来把数据从存储单元读出并装入寄存器, store 指令用来把寄存器的内容写入存储单元。显然, 这种风格的指令集中每条指令的功能强弱比较一致, 可以保证除 load 和 store 指令外的其他指令 (如运算指令) 在执行阶段都不访问存储器, 这样, 可把 load 和 store 指令的地址计算和运算指令的执行步骤规整在同一个流水段中, 有利于减少操作步骤, 以规整流水线。在 IA-32 这种非 load/store 型指令系统中, 运算类指令的操作数可以是存储器数据, 这样, 在指令执行过程中, 需要有存储器地址计算、存储器读和写以及运算等, 因而这类指令的执行要多出一些功能段, 与简单指令的功能段划分相差很大, 不利于流水线的规划。例如, IA-32 中的简单指令可以是寄存器之间传送 (如 “mov %ebx, %eax” 指令), 最多用三个功能段就可以完成, 而有的指令要先按照复杂的寻址方式计算存储单元地址, 然后再取出存储单元中的内容, 与某个寄存器内容进行运算, 最后再写到存储单元中 (如 “add %eax, 4(%ebp, %ebx, 4)” 指令), 因此, 至少要用 6 个功能段才能完成, 其中有两个功能段都需要访问存储单元。

第四, 为了便于以流水线方式执行指令, 数据和指令在存储器中要 “对齐” 存放。这样, 有利于减少访存次数, 使所需数据在一个流水段内就能从存储器中得到。

总之, 指令集体系结构的规整、简单和一致等特性有利于指令的流水线执行。

5.3.3 CISC 和 RISC 风格指令集

显然, 像 IA-32 这种指令格式不规整、寻址方式复杂多样、指令功能强弱不均的指令集来说, 用流水线方式来实现比较困难。Intel 为了保持市场占有率, 必须保持指令系统的兼容, 因而不能改变其指令系统设计风格, 但明摆着用流水线方式执行指令肯定比串行执行方式好, 那该怎么办呢? Intel 采用了一种称为 “CISC 壳、RISC 核” 的设计策略, 这里的 CISC 和 RISC 是指两种不同的指令设计风格。也就是说 IA-32 架构在指令集体系结构 (ISA) 层面是 CISC 风格的指令系统,

但是在执行指令的微体系结构层面采用的是 RISC 风格的微操作流水线执行机制。它将一条复杂的 CISC 风格的机器指令分解为多个 RISC 风格的微操作，对于每个微操作再用流水线方式来实现。

那么，CISC 和 RISC 具体是指什么呢？

1. CISC 风格指令系统

随着 VLSI 技术的迅速发展，计算机硬件成本不断下降，软件成本不断上升。为此，人们在设计指令系统时增加了越来越多功能强大的复杂指令，以使机器指令的功能接近高级语言语句的功能，给软件提供较好的支持。例如，VAX 11/780 指令系统包含了 16 种寻址方式、9 种数据格式、303 条指令，而且一条指令包含 1~2 个字节的操作码和后续 N 个操作数说明符，而一个操作数说明符的长度可达 1~10 个字节。我们称这类计算机为复杂指令集计算机（Complex Instruction Set Computer，简称 CISC）。

CISC 指令系统设计的主要特点如下。

- ① 指令系统复杂。指令条数多，寻址方式多，指令格式多而复杂，指令长度可变，操作码长度可变。
- ② 指令周期长。绝大多数指令需要多个时钟周期才能完成。
- ③ 相关指令会产生显式的条件码，存放在专门的标志寄存器（或称状态寄存器）中，可用于条件转移和条件传送等指令。
- ④ 指令周期差距大。各种指令都能访问存储器，有些指令还需要多次访问存储器，使得简单指令和复杂指令所用的时钟周期数相差很大，不利于指令流水线的实现。
- ⑤ 难以进行编译优化。由于编译器可选指令序列增多，使得目标代码组合增加，从而增加了目标代码优化的难度。

复杂指令系统使得其实现越来越复杂，不仅增加了研制周期和成本，而且难以保证正确性，甚至因为指令太复杂而无法采用硬连线路控制器，只能采用慢速的微程序控制器，从而降低了系统性能。

小贴士

控制器是整个 CPU 的指挥控制中心，也称为控制部件、控制逻辑或控制单元，其作用是对指令进行译码，将译码结果与状态/标志信号和时序信号等进行组合，产生指令执行过程中所需的控制信号。

控制信号被送到 CPU 内部或通过系统总线送到主存或 I/O 模块。送到 CPU 内部的控制信号用于控制 CPU 内部数据通路的执行，送到主存或 I/O 模块的信号控制 CPU 和主存之间或者 CPU 和 I/O 模块之间的信息交换。指令的执行过程就是数据通路中信息的流动过程。数据通路中信息的流动受控制信号的控制。不同的指令得到不同的控制信号取值，以规定数据通路完成不同的信息流动过程。各个功能部件中都有一些控制点，这些控制点接收不同的控制信号，使得功能部件完成不同的操作。例如：功能部件 ALU 的操作控制点可以接收“add”、“sub”、“and”、“or”、“mov”等不同的控制信号，以控制 ALU 完成加、减、与、或、传送等不同的操作。

根据不同的控制描述方式，可以有硬连线路控制器和微程序控制器两种实现方式。

硬连线路控制器的基本实现思路是，将指令执行过程中每个时钟周期所包含的控制信号取值组合看成一个状态，每来一个时钟周期，控制信号会有一组新的取值，也就是一个新的状态，这样，所有指令的执行过程就可以用一个有限状态转换图来描述。实现时，用一个组合逻辑电路（一般为 PLA 电路）来生成控制信号，用一个状态寄存器实现状态之间的转换。

微程序控制器的基本实现思路是，仿照程序设计方法，将每条指令的执行过程用一个**微程序**来表示，将指令执行过程中每个时钟周期所包含的控制信号取值组合看成是由多个微命令组成的一条**微指令**，每条微指令实际上就是一个 0/1 序列，一个微程序由若干条微指令组成。每个控制信号对应一个**微命令**，控制信号取不同的值，就发出不同的微命令。指令执行时，先找到对应的第一条微指令，然后按照特定的顺序取出后续的微指令执行。每来一个时钟周期，执行一条微指令。实现时，每条指令对应的微程序事先存放在一个只读存储器（称为**控制存储器**，简称**控存**）中，用一个 PLA 电路或 ROM 来生成每条指令对应的微程序的第一条微指令地址，用相应的微程序定序器来控制微指令执行流程。微程序定序器的实现有**计数器法**和**断定法**（即**下址字段法**）两种。

显然，硬连线路控制器速度快，适合于实现简单、规整的指令系统，如 MIPS 指令系统。硬连线路控制器是一个多输入/多输出的复杂、巨大的网状逻辑，对于 CISC 这种复杂指令系统来说，由于其控制逻辑极其复杂，因而实现起来非常困难，而且维护、扩充和修改都不容易，像 VAX11/780 这种极其复杂的指令系统，甚至都可能无法用有限状态机描述。而微程序控制器因为采用软件设计思想，不管指令多复杂，只要事先将其包含的操作所用的控制信号存储在控存中，就可以在指令执行时将控制信号取出，以控制指令的执行。因而，CISC 指令系统大多用微程序控制器实现。

对大量典型的 CISC 程序调查结果表明，各种指令的使用频率相当悬殊，最常使用的是只占指令系统 20% 的一些简单指令，它们占程序代码的 80% 以上，而需要大量硬件逻辑支持的复杂指令在程序中的出现频率却很低，造成了硬件资源的大量浪费。在微程序控制的计算机中，占程序中指令总数 20% 的最复杂的指令占用了微程序控制存储器容量的 80%。因此，1975 年 IBM 公司开始研究指令系统的合理性问题，John Cocke 领导的一个研究小组提出了**精简指令集计算机**（Reduced Instruction Set Computer，简称 RISC）的概念。

2. RISC 风格指令系统

RISC 的着眼点不是简单地放在简化指令系统上，而是通过简化指令系统使计算机结构更加简单合理，从而提高机器的性能。与 CISC 相比，RISC 指令系统的主要特点如下。

- ① 指令数目少。只包含使用频度高的简单指令。
- ② 指令格式规整。寻址方式少，指令格式少，指令长度一致，指令中操作码和寄存器编号等位置固定，便于取指令、指令译码以及提前读取寄存器内容等。
- ③ 采用 load/store 型指令设计风格。一条指令的执行阶段最多只有一次存储器访问操作。

④ 采用流水线方式执行指令。规整的指令格式有利于采用流水线方式执行，除 load/store 指令外，其他指令都只需一个或小于一个时钟周期就可完成，指令周期短。

⑤ 采用大量通用寄存器。编译器可将更多的局部变量分配到寄存器中，并且在过程调用时通过寄存器进行参数传递而不是通过栈进行传递，以减少访存次数。

⑥ 采用硬连线控制器。指令少而规整使得控制器的实现变得简单，可以不用或少用微程序控制器。

⑦ 实现细节对机器级程序可见。例如，有些 RISC 机器禁止一些特殊的指令序列，有些则规定条件转移指令后面必须填充若干条必须执行的指令等，这些都给编译器的设计和优化设定了相应的约束条件。

由于 RISC 指令系统简单，所以其 CPU 的控制逻辑大大简化，芯片上可设置更多的通用寄存器，也可以采用速度较快的硬连线控制器，且更适合于采用指令流水技术，这些都可以使指令的执行速度进一步提高。指令数量少，固然使编译工作量加大，但由于指令系统中的指令都是精选的，编译时间少，反过来对编译程序的优化又是有利的。

20 世纪 70 年代中期，IBM 公司、斯坦福大学、加州大学伯克利分校等机构先后开始对 RISC 技术进行研究，其成果分别用于 IBM、SUN、MIPS 等公司的产品中，如加州大学伯克利分校的 RISC I、斯坦福大学的 MIPS、IBM 公司的 IBM 801 相继宣告完成，这些机器被称为第一代 RISC 机。到 20 世纪 80 年代中期，RISC 技术蓬勃发展并广泛使用，先后出现了 PowerPC、MIPS、Sun SPARC、Compaq Alpha 等高性能 RISC 芯片以及相应的计算机。这时不少 RISC 的支持者开始对传统的 CISC 计算机（如 VAX、Intel、IBM 大型机）进行攻击，认为未来的发展非 RISC 莫属。当然，这也遭遇到计算机体系结构主流派的反对，这种争论延续了数年。

虽然 RISC 技术在性能上有优势，但最终 RISC 机并没有在市场上占优势，反而 Intel 一直保持处理器市场的较大份额，这是为什么呢？其原因主要有两点：首先，因为软件的向后兼容性，许多用户先期花了很多钱投资购买了在 Intel 系列机上开发的软件，如果换成 RISC 机，就意味着所有软件要重新投资；其次，随着处理器速度和芯片密度等的不断提高，RISC 系统也日趋复杂，而 CISC 采用了部分 RISC 技术，使其性能更加提高，例如，Intel 许多处理器的微体系结构采用了由硬件动态地将 CISC 指令翻译成类 RISC 指令的微操作，然后由流水线来执行微操作的做法。虽然这种混合方案不如纯 RISC 方案速度快，但是它却能在保证软件兼容的前提下达到具有较强竞争力的整体性能，而且 RISC 架构将许多底层实现细节暴露给机器级程序的思想也被证明是不明智的做法。不过，随着后 PC 时代的到来，个人移动设备的使用和嵌入式系统的应用越来越广泛，像 ARM 处理器等这些采用 RISC 技术的产品又迎来了新的机遇，目前在嵌入式系统中占有绝对优势。

5.3.4 指令流水线的实现

假定 MIPS 指令系统的执行微结构采用如图 5.14 所示的流水线数据通路，分成以下 5 个流水段。

取指 (Ifetch)：组合逻辑 A 中有指令存储器，该阶段实现取指令功能。

译码/取数 (Reg/Dec)：组合逻辑 B 中有寄存器堆，该阶段读取寄存器内容并对指令

译码。

执行 (Exec): 组合逻辑 C 中有 ALU, 该阶段实现算术和逻辑运算操作。

访存 (Mem): 组合逻辑 D 中有数据存储器, 该阶段实现存储器读或写操作。

写回 (Write): 组合逻辑 E 中有多路选择器, 该阶段选择存入寄存器的结果并将其在寄存器输入端稳定保持一段建立时间。

因为指令译码结果在第二个阶段才能得到, 因而前两个功能段是每条指令的公共操作, 所有指令在这两个流水段中的执行过程是完全一样的, 后面三个流水段根据不同指令译码结果进行不同的操作。

小贴士

对于 MIPS 指令系统, 每条指令前两个功能段的操作都一样。其中, Ifetch 段用于取指令并计算 $PC + 4$; Reg/Dec 段用于读取 rs 和 rt 寄存器的内容并对指令操作码进行译码, 而后面三个功能段随各指令功能的不同而不同。典型 MIPS 指令在 5 段流水线数据通路中的执行过程以及功能段的划分说明如下。

R 型指令 (汇编形式如 “sub rd, rs, rt”) 涉及在 ALU 中对 rs 和 rt 内容的运算, 并把 ALU 运算的结果送目的寄存器 rd。因此, 在 Ifetch 和 Reg/Dec 两个公共的功能段后, Exec 功能段在 ALU 中计算; Mem 功能段为空操作; Write 功能段将 ALU 运算结果写寄存器 rd。

I 型带立即数的运算类指令 (汇编形式如 “addi rt, rs, imm16”) 涉及对 16 位立即数 imm16 进行符号扩展或零扩展, 然后和 rs 的内容进行运算, 最终把 ALU 的运算结果送目的寄存器 rt。显然, I 型运算类指令的功能段划分与 R 型指令类似。

lw 指令 (汇编形式为 “lw rt, imm16(rs)”) 的功能为 “ $R[rt] \leftarrow M[R[rs] + \text{SignExt}(\text{imm16})]$ ”, 即把存储单元的内容装入寄存器中。除两个公共的功能段外, Exec 功能段在 ALU 中计算存储单元地址; Mem 功能段从存储单元中读数据; Write 功能段将数据写寄存器 rt。

beq 指令 (汇编形式为 “beq rs, rt, imm16”) 是一条分支指令 (即条件转移指令), 指令的功能如下:

```
if R[rs] = R[rt]
    PC ← PC + 4 + 4 × imm16
else
    PC ← PC + 4
```

除两个公共的功能段外, Exec 功能段对 $R[rs]$ 和 $R[rt]$ 做减法, 同时计算转移目标地址 $PC + 4 + 4 \times \text{imm16}$; Mem 功能段根据 $R[rs]$ 和 $R[rt]$ 是否相等来确定是否将转移目标地址送 PC; Write 功能段为空操作。

图 5.15 是流水线在理想情况下的正常执行过程示意图, 实际上流水线的执行可能会被破坏。

如图 5.15 所示, 假定某段时间执行的指令序列为 slt、lw、beq、sub、or、addi、…、add, 其中, slt、sub、or 和 add 是 R 型指令, addi 为 I 型运算类指令。该指令序列在流水线中执行

时,可能会遇到一些情况,使流水线无法正确、按时执行后续指令,从而引起流水线阻塞或停顿 (stall),我们称这种现象为流水线冒险 (hazard)。在图 5.15 中可能存在的流水线冒险包括结构冒险、数据冒险和控制冒险。

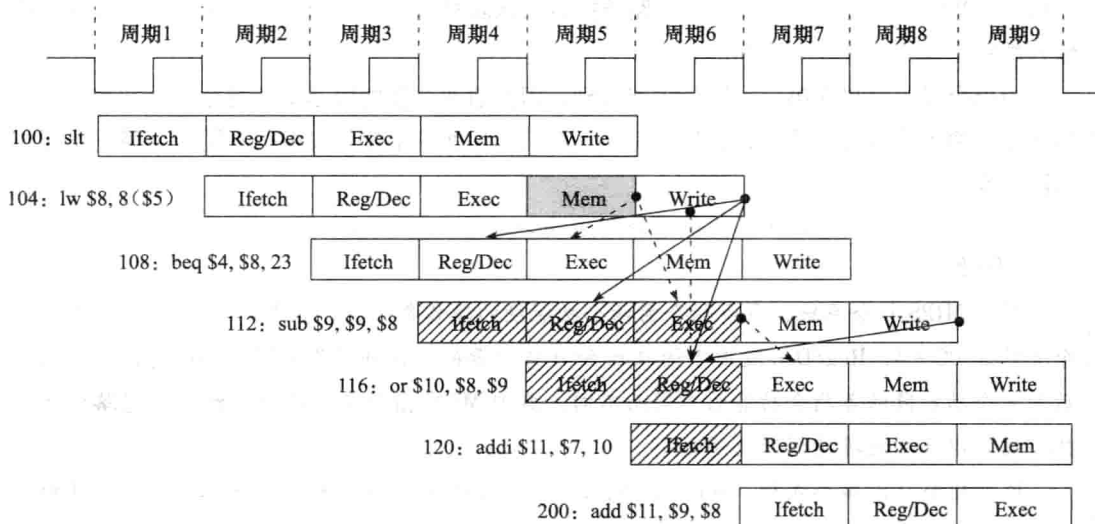


图 5.15 MIPS 流水线执行示意图

1. 结构冒险

结构冒险 (structural hazard) 也称为硬件资源冲突 (hardware resource conflict), 引起结构冒险的原因在于同一个部件同时被不同指令所用, 也就是说它是由硬件资源竞争造成的。

对于图 5.14 所示的 MIPS 流水线数据通路, 如果组合逻辑 A 中的指令存储器和组合逻辑 D 中的数据存储器是同一个存储器的话, 则图 5.15 中地址为 104 的 lw 指令在 Mem 阶段取数据的同时, 随后第三条指令 or 正好在取指令阶段 Ifetch, 此时两条指令都要访问同一个存储器, 因而发生访存冲突, 引起结构冒险。

解决结构冒险的策略有两个方面: ① 规定一个部件每条指令只能使用一次, 且只能在特定阶段使用。② 通过设置多个独立的部件来避免资源冲突。例如, 对于访存冲突, 可把指令存储器和数据存储器分开, 也即组合逻辑 A 中有一个独立的指令存储器, 组合逻辑 D 中有一个独立的数据存储器, 并且指令存储器只能在 Ifetch 阶段使用, 数据存储器只能在 Mem 阶段使用。事实上, 现代计算机都引入了高速缓冲存储器 (cache), 而且一级 cache 中采用了数据 cache 和代码 cache 分开设置的方式, 这样就避免了访存冲突引起的结构冒险。

2. 数据冒险

数据冒险 (data hazard) 也称为数据相关 (data dependency)。引起数据冒险的原因在于, 后面指令用到前面指令的运算结果时, 前面指令的结果还没产生。例如, 图 5.15 中实线箭头处所示的是数据冒险现象, 即指令 lw 和 beq 之间关于寄存器 \$8、lw 和 sub 之间关于寄存器 \$8、lw 和 or 之间关于寄存器 \$8、sub 和 or 之间关于寄存器 \$9, 这些都是数据冒险的情况。

指令 “lw \$8, 8(\$5)” 的功能为 $R[8] \leftarrow M[R[5] + 8]$ 。从图 5.15 可以看出, 从存储器读出的数据在第 6 个时钟周期写入到 8 号寄存器 \$8, 因此它第 7 个时钟周期开始才能被读出

使用,可是,指令 beq 和 sub 分别在第 4 和第 5 时钟周期就要读 \$8,如果不阻塞流水线或采取相应措施的话,读到的就是 \$8 的旧值,从而产生数据冒险。

解决数据冒险的策略有以下多种。

① 最简单的是由编译器在数据相关的指令之间加若干 nop 指令 (即空操作指令,除修改 PC 外其他什么操作都不做)。例如,在 beq 指令前加三条 nop 指令,使得 beq 指令取数阶段延迟到第 7 周期执行。显然这种方式简化了硬件,但增加了时间开销 (执行 nop 指令的时间) 和空间开销 (nop 指令所占的空间)。

② 采用数据转发 (forwarding) 机制,在数据通路中一旦产生运算结果或一旦存储器读出数据,就把它们通过一条旁路 (bypass) 直接送到相关后续指令在 Exec 阶段的 ALU 输入端,这样,后续相关指令不必等到前面指令把结果写到寄存器后再从寄存器读出。

③ 对于像图 5.15 中指令 lw 和 beq 之间的这种数据相关,无法通过转发来解决,因为 lw 指令在 Mem 阶段结束才能取到数据,此时已经是第 5 时钟周期结束,而 beq 指令在 Exec 阶段中的 ALU 输入端应该在第 5 时钟周期开始时就需要操作数,显然,这种情况下数据来不及转发。这种 load 指令装入的数据在随后一条指令就需要使用的情况称为load-use 数据冒险,它不能用转发来解决。这种情况下,可通过硬件阻塞的方式 (插入气泡) 来延迟 load 指令随后一条指令的执行。有关 load-use 数据相关检测和硬件阻塞机制已经超出本书范围,请参看其他相关资料。

④ 对于像图 5.15 中 lw 和 or 之间关于寄存器 \$8 这样的数据相关问题,则可以通过对通用寄存器的读写操作进行特殊处理,保证在一个时钟的前半周期进行寄存器写,在后半周期进行寄存器读,这样, lw 指令在第 6 时钟周期的前半周期将从存储器取出的内容写入寄存器 \$8,而 or 指令则在后半周期从寄存器 \$8 中取数。

图 5.16 给出了一个对寄存器 \$1 的内容进行转发执行的例子,该例中所有数据相关都可用转发解决。

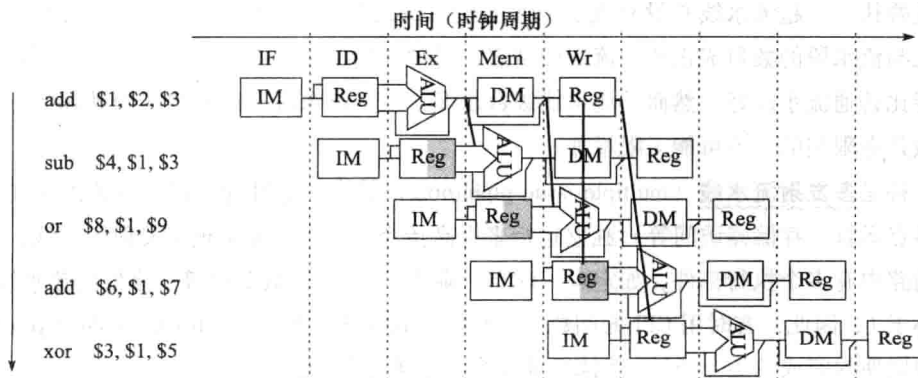


图 5.16 利用转发解决数据冒险的例子

3. 控制冒险

正常情况下,指令在流水线中总是按顺序执行,当遇到改变指令执行顺序的情况时,流水线中指令的正常执行会被阻塞。这种由于发生了指令执行顺序改变而引起的流水线阻塞称为控

制冒险。各类转移指令（包括条件转移、无条件转移、调用、返回指令等）以及第7章介绍的异常和中断等事件都会改变指令执行顺序，因而都可能会引发控制冒险。

beq 在 Exec 阶段对 $R[rs]$ 和 $R[rt]$ 做减法，同时计算转移目标地址；在 Mem 阶段根据 $R[rs]$ 和 $R[rt]$ 是否相等确定是否将转移目标地址送 PC。若 $R[rs] = R[rt]$ ，则在图 5.15 所示流水线中，beq 指令在第 6 时钟周期将转移目标地址 $PC + 4 + 4 \times 23 = 104 + 96 = 200$ 送到 PC 的输入端，在第 7 时钟信号到来后，取出 200 号单元的指令“add \$11, \$9, \$8”执行。此时，紧跟在 beq 后面的三条指令已在流水线中被执行了一部分（图 5.15 中加斜线的流水段）。显然，正确的执行流程应该是 beq 指令执行完后直接转移到 200 号单元处执行，因此，如果不采取相应措施，则指令流水线的执行便发生问题。通常把由于流水线阻塞而带来的延迟执行周期数称为延迟损失时间片。显然，这里的延迟损失时间片 $C = 3$ 。

由于分支指令而引起的控制冒险也称为分支冒险（branch hazard）。对于分支冒险，可采用和前面解决数据冒险一样的硬件阻塞方式（插入气泡）或软件阻塞方式（插入空操作指令）。也即，假设延迟损失时间片为 C ，则在数据通路中检测到分支指令时，就在分支指令后插入 C 个气泡，或在编译时在分支指令后填入 C 条 nop 指令。

不过，插入气泡和插入空操作指令这两种都是消极的方式，效率较低。结合分支预测可以降低由于分支冒险带来的时间损失，分支预测有静态预测和动态预测两种。此外，延迟分支方式也可部分解决分支冒险问题。有关这些内容已经超出本书讨论的范围，请参看其他相关资料。

5.3.5 高级流水线实现技术

高级流水线技术充分利用指令级并行（Instruction-Level Parallelism，简称为 ILP）来提高流水线的性能。有两种增加指令级并行的策略。

一种是超流水线（super-pipelining）技术，通过增加流水线级数来使更多的指令同时在流水线中重叠执行。超流水线并没有改变 CPI 的值，CPI 还是 1，但是，因为理想情况下流水线的加速比与流水段的数目成正比，流水段越多，时钟周期越短，指令吞吐率越高，所以超流水线的性能比普通流水线好。然而，流水线级数越多，用于流水段寄存器的开销就越大，因而流水线级数是有限制的，不可能无限增加。

另一种是多发射流水线（multiple issue pipelining）技术，通过同时启动多条指令（如整数运算、浮点运算、存储器访问等）独立运行来提高指令并行性。要实现多发射流水线，其前提是数据通路中有多个执行部件，如定点、浮点、乘除、取数/存数部件等。多发射流水线的 CPI 能达到小于 1，因此，有时用 CPI 的倒数 IPC 来衡量其性能。IPC（Instructions Per Cycle）是指每个时钟周期内完成的指令条数。例如，4 路多发射流水线的理想 IPC 为 4。

实现多发射流水线必须完成以下两个任务：指令打包和冒险处理。指令打包任务就是能够将并行处理的多条指令同时发送到发射槽中，因此处理器必须知道每个周期能发射几条指令，哪些指令可以同时发射。这通过推测（speculation）技术来完成，可以由编译器或处理器通过猜测指令执行结果来调整指令执行顺序，使指令的执行能达到最大可能的并行。指令打包的决策依赖于“推测”的结果，主要根据指令间的相关性来进行推测，与前面指令不相关的指令

可以提前执行,例如,如果可以推测出一条 load 指令和它之前的 store 指令引用的不是同一个存储地址,则可以将 load 指令提前到 store 指令之前执行;也可对分支指令进行推测以提前执行分支目标处的指令。不过,推测仅是“猜测”,有可能推测结果是错误的,故需有推测错误检测和回退机制,在检测到推测错误时,能回退被错误执行的指令。因此,错误推测会导致额外开销。推测需要结合软件推测和硬件推测来进行,软件推测指编译器通过推测来静态重排指令,此种推测一定要正确,而硬件推测指处理器在程序执行过程中通过推测来动态调度指令。

根据推测任务主要由编译器静态完成还是由处理器动态执行,可将多发射技术分为两类:静态多发射和动态多发射。

静态多发射主要通过编译器静态推测来辅助完成“指令打包”和“冒险处理”。指令打包的结果可看成将同时发射的多条指令合并到一个长指令中。通常将一个时钟周期内发射的多个指令看成一条多个操作的长指令,称为一个“发射包”。所以,静态多发射指令最初被称为“超长指令字”(Very Long Instruction Word, VLIW),采用这种技术的处理器被称为VLIW 处理器。Intel 的 IA-64 架构采用这种方法,Intel 称其为 EPIC(Explicitly Parallel Instruction Computer, 显式并行指令计算机)。

动态多发射由处理器硬件动态进行流水线调度来完成“指令打包”和“冒险处理”,能在一个时钟周期内执行一条以上指令。采用动态多发射流水线技术的处理器称为超标量(super-scalar)处理器。在简单的超标量处理器中,指令按顺序发射,每个周期由处理器决定是发射一条或多条指令,显然,在这种处理器上要达到较好的性能,很大程度上依赖于编译器。为了更好地发挥超标量处理器的性能,多数超标量处理器都结合动态流水线调度(dynamic pipeline scheduling)技术,处理器通过指令相关性检测和动态分支预测等手段,投机性地不按指令顺序执行,当发生流水线阻塞时,根据指令的依赖关系,动态地到后面找一些没有依赖关系的指令提前执行。这种指令执行方式称为乱序执行(out-of-order execution)。

有关高级流水线的实现技术的内容已经超出本书讨论范围,请参看其他相关资料。

5.4 小结

本章主要介绍程序中一条一条指令在机器上的执行过程以及执行指令的数据通路的基本工作原理和基本实现原理。主要内容包括 CPU 的主要功能和内部结构、指令的执行过程、数据通路的基本组成、数据通路中信息的流动过程、内部异常和外部中断的基本概念、流水线的基本实现原理等。

CPU 的基本功能是周而复始地执行指令,并处理内部异常和外部中断。CPU 最基本的部分是数据通路和控制单元。数据通路中包含组合逻辑元件和存储信息的状态元件。组合逻辑元件(如加法器、ALU、扩展器、多路选择器以及状态元件的读操作逻辑等)用于对数据进行处理;状态元件包括触发器、寄存器和存储器等,用于对指令执行的中间状态或最终结果进行存储。控制单元对取出的指令进行译码,与指令执行得到的条件标志或当前机器的状态、时序信号等组合,生成对数据通路进行控制的控制信号。

指令执行过程主要包括取指、译码、取数、运算、存结果。通常把取出并执行一条指令的

时间称为指令周期，它由机器周期或直接由时钟周期组成。现代计算机已经没有机器周期的概念，其每个指令周期直接由时钟周期（节拍）组成。时钟信号是 CPU 中用于控制信号同步的信号。

每条指令的功能不同，因此每条指令执行时数据在数据通路中所经过的部件和路径也可能不同。但是，每条指令在取指令阶段都一样。

早期计算机中数据通路采用总线方式，通过 CPU 的内总线把 CPU 中的通用寄存器、ALU、暂存器、指令寄存器等互连，有单总线、双总线 and 三总线结构数据通路。

现代计算机的数据通路都采用流水线方式实现，将每条指令的执行过程分解成功能段相同的几个流水段，每个流水段的执行时间也被设置成完全相同。流水线方式下，同时有多条指令重叠执行，因此程序的执行时间比串行执行方式下缩短很多。指令流水线在有些情况下会发生流水线冒险，包括结构冒险、数据冒险和控制冒险三类。

习题

1. 给出以下概念的解释说明。

指令周期	机器周期	控制信号	控制部件
执行部件	操作元件	状态元件	多路选择器
程序计数器 (PC)	指令寄存器 (IR)	指令译码器 (ID)	硬连线控制器
微程序控制器	控制存储器 (CS)	微指令	微程序
内部异常	外部中断	指令流水线	指令吞吐率
流水段寄存器	流水线冒险	结构冒险	数据冒险
控制冒险	流水线阻塞	空操作	转发（旁路）
延迟时间损失片	静态多发射	动态多发射	超流水线
超长指令字 (VLIW)	超标量流水线	动态流水线调度	乱序执行

2. 简单回答下列问题。

- (1) CPU 的基本组成和基本功能各是什么？
- (2) 如何控制一条指令执行结束后能够接着另一条指令执行？
- (3) 通常一条指令的执行要经过哪些步骤？每条指令的执行步骤都一样吗？
- (4) 取指令部件的功能是什么？控制器的功能是什么？
- (5) 为什么按异步方式访问存储器时需要 WMFC 信号，而按同步方式访存时无需 WMFC 信息？
- (6) 硬连线控制器和微程序控制器的特点各是什么？
- (7) 为什么 CISC 大多用微程序控制器实现，RISC 大多用硬连线控制器实现？
- (8) 流水线方式下，一条指令的执行时间缩短了还是加长了？程序的执行时间缩短了还是加长了？
- (9) 具有什么特征的指令集易于实现指令流水线？

3. 假定图 5.8 中总线传输延迟和 ALU 运算时间分别是 20ps 和 200ps，寄存器建立时间为 10ps，

寄存器保持时间很小,可忽略不计,寄存器的锁存延迟(Clk-to-Q)为4ps,控制信号生成的延迟时间(Clk-to-Signal)为7ps,三态门接通时间为3ps,则从当前时钟到达开始算起,完成以下操作的最短时间是多少?

(1) 将数据从一个寄存器传送到另一个寄存器。

(2) 将程序计数器PC加1。

4. 图5.17给出了某CPU内部结构的一部分, MAR和MDR直接连到存储器总线(图中省略)。在总线A和B之间的所有数据传送都需经过算术逻辑部件ALU。ALU的部分控制信号及其功能如下:

MOVa:F = A; MOVb:F = B;

INCa:F = A + 1; INCb:F = B + 1;

DECa:F = A - 1; DECb:F = B - 1。

其中A和B是ALU的输入,F是ALU的输出。假定该CPU的指令系统中调用指令CALL占两个字,第一个字是操作码,第二个字给出子程序的起始地址,返回地址保存在主存的栈中,用SP指向栈顶,存储器按字编址,每次按同步方式从主存读取一个字,请写出读取并执行CALL指令所要求的控制信号序列(提示:当前指令地址已在PC中)。

5. 假定某计算机字长16位,CPU内部结构如图5.6所示,CPU和存储器之间采用同步方式通信,按字编址。指令采用定长指令字格式,由两个字组成,第一个字指明操作码和寻址方式,第二个字包含立即数imm16。若一次存储器访问所用时间为两个CPU时钟周期,每次存储器访问存取一个字,取指令阶段第二次访存将imm16取到MDR中,请写出下列指令在指令执行阶段(不考虑取指令阶段)的控制信号序列,并说明需要几个时钟周期。

(1) 将立即数imm16加到寄存器R1中,此时,imm16为立即操作数。即

$$R[R1] \leftarrow R[R1] + \text{imm16}$$

(2) 将地址为imm16的存储单元的内容加到寄存器R1中,此时,imm16为直接地址。即

$$R[R1] \leftarrow R[R1] + M[\text{imm16}]$$

(3) 将存储单元imm16的内容作为地址所指的存储单元的内容加到寄存器R1中。此时,imm16为间接地址。即

$$R[R1] \leftarrow R[R1] + M[M[\text{imm16}]]$$

6. 假定在一个如图5.14所示的5级流水线处理器中,各主要功能单元的操作时间如下:存储单元为200ps;ALU或加法器为150ps;寄存器堆读口或写口为50ps。回答下列问题。

- (1) 若执行阶段EXE所用的ALU操作时间缩短20%,则能否加快流水线执行速度?如果能的话,能加快多少?如果不能的话,为什么?
- (2) 若ALU操作时间增加20%,对流水线的性能有何影响?
- (3) 若ALU操作时间增加40%,对流水线的性能又有何影响?

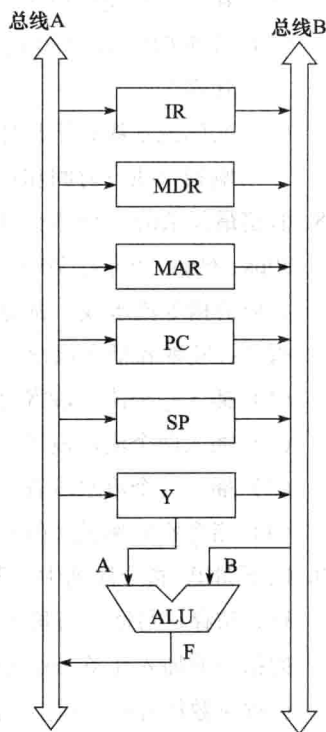


图5.17 题4用图

7. 假定某计算机工程师想设计一个新的 CPU，一个典型程序的核心模块有一百万条指令，每条指令执行时间为 100ps。请问：
- (1) 在非流水线处理器上执行该程序需要花多长时间？
 - (2) 若新 CPU 采用 20 级流水线，执行上述同样的程序，理想情况下，它比非流水线处理器快多少？
 - (3) 实际流水线并不是理想的，流水段之间的数据传送会有额外开销。这些开销是否会影响指令执行时间和指令吞吐率？
8. 假定最复杂的一条指令所用的组合逻辑分成六部分，依次为 A ~ F，其延迟分别为 80ps、30ps、60ps、50ps、70ps、10ps。在这些组合逻辑块之间插入必要的流水段寄存器就可实现相应的指令流水线，寄存器延迟为 20ps。理想情况下，以下各种方式所得到的时钟周期、指令吞吐率和指令执行时间各是多少？应该在哪里插入流水段寄存器？
- (1) 插入一个流水段寄存器，得到一个两级流水线。
 - (2) 插入两个流水段寄存器，得到一个 3 级流水线。
 - (3) 插入三个流水段寄存器，得到一个 4 级流水线。
 - (4) 指令吞吐率最大的流水线。
9. 以下 MIPS 指令序列中，哪些指令对之间发生数据相关？假定采用“取指、译码/取数、执行、访存、写回”五段流水线方式，那么不采用“转发”技术的话，需要在发生数据相关的指令前加入几条 nop 指令才能使这段程序避免数据冒险？如果采用“转发”是否可以完全解决数据冒险？（注：指令中的 t1 ~ t2、s0 ~ s3 是 MIPS 寄存器名，//后是注释。）

```

addu    $s2, $s1, $s0           // R[$s2] ← R[$s1] + R[$s0]
addu    $t2, $s2, $s2           // R[$t2] ← R[$s2] + R[$s2]
lw      $t1, 0($t2)             // R[$t1] ← M[R[$t2] + 0]
add     $s3, $t1, $t2           // R[$s3] ← R[$t1] + R[$t2]

```

10. 假设一个程序的 MIPS 指令序列为“lw, add, lw, add, …”。add 指令仅依赖它前面的 lw 指令，而 lw 指令也仅依赖它前面的 add 指令，寄存器写口和寄存器读口分别在一个时钟周期的前、后半周期内独立工作。回答下列问题。
- (1) 在带转发的 5 级流水线中执行该程序，其 CPI 为多少？
 - (2) 在不带转发的 5 级流水线中执行该程序，其 CPI 为多少？
11. 假定在一个带转发的五段流水线中执行以下 MIPS 程序段，则怎样调整指令序列使其性能达到最好？

```

loop: lw    $2, 100($6)          // R[$2] ← M[R[$6] + 100]
      add   $2, $2, $3           // R[$2] ← R[$2] + R[$3]
      lw    $3, 200($7)         // R[$3] ← M[R[$7] + 200]
      add   $6, $4, $7           // R[$6] ← R[$4] + R[$7]
      sub   $3, $4, $6           // R[$3] ← R[$4] - R[$6]
      lw    $2, 300($8)         // R[$2] ← M[R[$8] + 300]
      beq   $2, $8, loop        // if R[$2] = R[$8] goto loop

```